Product Backlog: AEGIS

This document provides the user stories that define the functional specifications of the platform. These stories form the foundation of the product backlog and will be continuously fine tuned through clear acceptance criteria for sprint planning.

User Personas

- **Recruiter:** The Admin or technical team member who creates, manages, and reviews candidate assessments.
- **Candidate:** The job applicant completing the technical evaluation.

Epic 1: Question & Assessment Management (Recruiter)

US1.1: AI-Resistant Question

User Story:

As a recruiter, I want to generate modified versions of coding questions that are resistant to AI-generated answers, so I can evaluate a candidate's true problem-solving abilities.

Acceptance Criteria:

- **Criteria 1.1.1: Transformation Rule**
    - **Given:** An existing coding question.
    - **When:** The recruiter requests an AI-resistant modification.
    - **Then:** The system must transform the question while preserving the original programming concept and difficulty level.

- **Criteria 1.1.2: Review Interface**
    - **Given:** A generated question.
    - **When:** Displayed to the recruiter.
    - **Then:** The platform must provide options to regenerate, accept, or discard the question.

- **Criteria 1.1.3: Manual Question Validation:**
    - **Given:** A generated answer from Gemini to that generated adversarial question.
    - **When:** The Gemini AI provides it.
    - **Then:** The recruiter will compare the answer it has received from Gemini with the stored answer for that original question for question validation.

US1.2: Question Bank Management

User Story:

As a recruiter, I want to manage and search coding questions in a question bank, so I can select suitable questions when creating assessments.

Acceptance Criteria:

- **Criteria 1.2.1: Search & Multi-Filter**
  - **Given:** The Question Bank dashboard.
  - **When:** A recruiter enters text in the search bar or applies a filter for difficulty, language and category
  - **Then:** The question list must update instantly to display only matching records.

- **Criteria 1.2.2: CRUD Operations**
  - **Given:** The Question Bank dashboard.
  - **When:** A recruiter to create, read, update, and delete any question.
  - **Then:** The system must execute the requested operation and update the repository state in the database.

- **Criteria 1.2.3: Categorization**
  - **Given:** A question details page.
  - **When:** A recruiter adds custom text strings into the tags field and saves.
  - **Then:** The system must associate those custom strings with a question.

US1.3: Assessment Creation & Assignment

User Story:

As a recruiter, I want to create and assign assessments by selecting adversarial questions and adjusting assessment settings, so candidates can complete structured technical assessments.

Acceptance Criteria:

- **Criteria 1.3.1.a: Successful Validation**
  - **Given:** A recruiter is on the assessment creation form.
  - **When:** They fill out all mandatory fields (Title, Description, Time Limit) and click “Save”.
  - **Then:** The system must successfully save the assessment.

- **Criteria 1.3.1.b: Failed Validation Error**
    - **Given:** A recruiter is on the assessment creation form.
    - **When:** They leave a mandatory field empty and click “Save”.
    - **Then:** The system must block the save action and display an error message next to the missing field(s).

- **Criteria 1.3.2: Candidate Status Tracking**
    - **Given:** The recruiter is viewing the assessment tracking dashboard.
    - **When:** A candidate’s progress changes.
    - **Then:** The roster view must dynamically update that candidate’s status badge to match (Ready To Start, In Progress, or Completed).

Epic 2: Technical Assessment Environment (Candidate)

US2.1: Code Editor & Execution Engine

User Story:

As a candidate, I want to answer assessment questions using a code editor and test my solutions, so I can complete the assessment and verify my implementation.

Acceptance Criteria:

- **Criteria 2.1.1: IDE Presentation**
    - **Given:** The candidate opens an active assessment question.
    - **When:** The code editor activates.
    - **Then:** It must apply syntax highlighting, tab spacing, and bracket auto-closing adjusted to the selected programming language.

- **Criteria 2.1.2: On-Demand Code Execution**
    - **Given:** A candidate has written code in the text canvas.
    - **When:** They click the “Run Code” button.
    - **Then:** The backend engine must execute the code against the configured test cases in a Piston sandbox and display the execution results to the candidate.

US2.2: Automated Progress Saving

User Story:

As a candidate, I want my assessment progress and answers to be saved automatically or manually, so I don’t lose my work during an assessment.

Acceptance Criteria:

- **Criteria 2.2.1: Next Button Auto-Save**
    - **Given:** A candidate is completing an assessment.

    - **When:** They click the **"Next"** button.
    - **Then:** The system must save the candidate's current progress to the database before loading the next question.

- **Criteria 2.2.2: State Recovery on Re-Entry**
    - **Given:** A candidate loses their browser connection or crashes mid-test.
    - **When:** They re-open their unique assessment access link within the remaining exam window.
    - **Then:** The workspace must restore the exact assessment